

Crash-Recoverable Post-Quantum Key Migration with Disjunctive Hybrid Verification: Design, Implementation, and Formal Verification

Thomas W Rodriguez Jr.

Protocol Architect

Indianapolis, Indiana, United States

trodriguez@higgaion.io • <https://higgaion.io>

Abstract

The transition from classical elliptic-curve cryptography to post-quantum algorithms is an urgent infrastructure challenge, yet existing migration approaches require all-or-nothing upgrades, lack crash recovery, and mandate conjunctive verification that prevents incremental deployment. We present a PQC Wallet Migration Engine: a standalone C library implementing a four-state crash-recoverable migration state machine with write-ahead log (WAL) atomicity, disjunctive hybrid dual-signing, and multi-chain key import. The engine migrates keys through CLASSICAL, HYBRID, FINALIZING, and PQC_ONLY states, with each transition journaled for crash recovery. In HYBRID mode, dual classical/ML-DSA-87 signatures support disjunctive (accept-either) verification, enabling zero-downtime migration across networks of mixed-capability verifiers. We present a quad-tier formal verification campaign comprising TLA+ model checking (3 specifications, 16 invariants), CBMC bounded model checking (13 harnesses), Frama-C WP deductive verification (18 ACSL properties), and Coq mechanized proofs (101 theorems). The full test suite of 26 lifecycle tests executes in under 500 ms on commodity hardware.

Keywords: post-quantum cryptography, key migration, ML-DSA-87, ML-KEM-1024, formal verification, write-ahead log, hybrid signatures

1. Introduction

The cryptographic foundations of blockchain ecosystems and enterprise public-key infrastructure face an existential threat from quantum computing. Shor’s algorithm renders all deployed elliptic-curve cryptography (ECC), including secp256k1 (Bitcoin, Ethereum), NIST P-256/P-384 (TLS/PKI), and Ed25519 (Solana, Cardano, SSH), vulnerable to key recovery attacks on sufficiently powerful quantum hardware [1].

NIST has standardized replacements: ML-KEM-1024 (FIPS 203) for key encapsulation and ML-DSA-87 (FIPS 204) for digital signatures [2, 3]. Federal mandates including OMB Memorandum M-23-02, National Security Memorandum NSM-10, and Executive Order 14028 establish deadlines for migration [4]. However, the migration itself presents a systems engineering challenge that existing tools do not adequately address.

The core tension is between security continuity and operational availability. A wallet holding cryptographic assets cannot simply swap its key algorithm; doing so would invalidate all existing signatures and break interoperability with peers that have not yet upgraded. The migration must be gradual, crash-safe, and support heterogeneous verification during the transition period.

This paper presents a PQC Wallet Migration Engine that addresses five specific gaps in the current landscape:

1. **Crash-recoverable state transitions.** Each key’s migration state is journaled to a write-ahead log with CRC32 integrity checking and monotonic sequence numbers, providing atomic state transitions that survive process crashes and power failures.
2. **Disjunctive (OR-mode) hybrid verification.** Unlike IETF composite signatures [5] which require conjunctive (AND-mode) verification, our system accepts a message as valid if *either* the classical or PQC

signature verifies independently, enabling zero-downtime migration without coordinated network-wide upgrade.

3. **Multi-chain key import.** Format-specific adapters import keys from Bitcoin (WIF), Ethereum (hex with Keccak-256 address derivation), ECDSA (DER for P-256/P-384), and Ed25519 (Solana, Cardano) with a universal SHA3-256 key identifier that remains stable across the entire migration lifecycle.
4. **Pre-migration self-test gating.** An operation-specific cryptographic self-test (ML-DSA-87 sign/verify round-trip) gates each individual key’s transition, distinct from any module-level initialization self-test.
5. **Quad-tier formal verification.** The implementation is verified using TLA+ model checking, CBMC bounded model checking, Frama-C WP deductive verification, and Coq mechanized proofs.

Disclosure

The implementation was developed with AI coding assistants (Claude/Antigravity, Grok) under the direction of the author. All code, architectural decisions, verification strategies, and test designs were reviewed, directed, and validated by the author. This paper itself was drafted with AI assistance and reviewed by the author for accuracy. A provisional patent application covering this system was filed with the USPTO on March 9, 2026.

2. Related Work

IETF Composite Signatures. Draft-ietf-lamps-pq-composite-sigs [5] defines CompositeSignatureValue combining classical and PQC signature outputs with conjunctive verification: all components must pass. This prevents incremental deployment in heterogeneous networks where not all verifiers support PQC. Our disjunctive mode addresses this limitation.

IETF Hybrid Signature Taxonomy. Draft-ietf-pquip-hybrid-signature-spectrums [6] taxonomizes hybrid signature design goals but does not prescribe disjunctive verification as an explicit design point.

IETF Dual Certificates. Draft-yusef-tls-pqt-dual-certs [7] extends TLS 1.3 for dual classical/PQC certificate authentication using conjunctive verification.

IBM Quantum Safe Migration Tools. IBM’s Quantum Safe Migration Orchestrator (QSMO), Remediator, and Explorer provide enterprise PQC migration for TLS configurations, LDAP entries, and certificates [8]. The changeCipher utility offers resume-from-interruption capability. However, IBM tools operate at the application-configuration level and do not provide WAL-backed atomic state machine transitions for individual cryptographic key lifecycles.

Thales Luna HSM. Thales HSMs provide pre-rotation domain validation during key rotation [9]. This operational check validates domain functionality, not a cryptographic round-trip self-test gating individual key migration state transitions.

Quantum Resistant Ledger (QRL). QRL was built quantum-safe from genesis and migrates between PQC variants (XMSS to SPHINCS+) within the same paradigm (hash-based to hash-based) [10]. It does not import classical keys from external chains or migrate across paradigm boundaries (ECC to lattice-based).

IOTA Identity. IOTA abandoned Winternitz OTS for Ed25519 and is adding ML-DSA hybrid signatures via IOTA Identity 1.7 Beta [11]. IOTA provides no formal migration state machine, no WAL crash recovery, and hybrid signatures are SDK-level rather than protocol-mandated.

No identified prior work combines: (a) a crash-recoverable migration FSM with WAL atomicity; (b) disjunctive hybrid verification; (c) multi-chain key import; and (d) operation-specific PQC self-test gating.

3. System Design

3.1 Migration State Machine

The engine maintains each key in one of four states:

State	Value	Description
CLASSICAL	0	Only the original ECC key exists
HYBRID	1	Both classical and PQC keypairs are active
FINALIZING	2	PQC self-test passed; awaiting administrator confirmation
PQC_ONLY	3	Classical key material irreversibly erased

State transitions are strictly ordered:

- CLASSICAL → HYBRID (requires self-test pass)
- HYBRID → FINALIZING (forward progress)
- HYBRID → CLASSICAL (rollback)
- FINALIZING → PQC_ONLY (irreversible; requires key erasure)

The state machine is defined in `pqc_migration.c` as `migration_begin()`, `migration_finalize()`, and `migration_rollback()`. Each function enforces the transition preconditions and atomically updates state via the WAL.

3.2 Write-Ahead Log for Crash Atomicity

Every state transition is journaled to a persistent WAL file before modifying in-memory state, with one critical exception: during finalization, classical key material is erased *before* the WAL records the PQC_ONLY transition.

The WAL record is a fixed 101-byte packed binary structure (v2):

Field	Size	Description
Magic	8 bytes	0x0047494D5F474948
Version	1 byte	Protocol version (v2)
Operation	1 byte	0=import, 1=begin, 2=finalize, 3=rollback, 4=enter finalizing, 5=peer state update
Key ID	65 bytes	Universal SHA3-256 identifier
Target State	1 byte	Destination state value
Sequence	8 bytes	Monotonic counter
Peer State	1 byte	Remote shard's reported state (v2, op=5)
Peer Shard ID	4 bytes	Source shard identifier (v2, op=5)
Peer Updated At	8 bytes	Peer report timestamp (v2, op=5)
CRC32	4 bytes	Checksum over all preceding fields

The ordering during finalization is significant: WAL write occurs *after* key destruction. If the system crashes after erasure but before the WAL write, recovery restores the key to HYBRID state, but the classical key material is already gone. This is the correct behavior: the system fails safe by preventing recovery from resurrecting erased key material; the PQC keys remain usable in HYBRID mode.

On initialization, `migration_engine_init()` replays the WAL by validating each record's magic number and CRC32 checksum, applying only valid transitions, and truncating the WAL after successful replay.

3.3 Disjunctive Hybrid Verification

In HYBRID state, `migration_hybrid_sign()` produces two signatures over the same message:

1. A classical signature using the original ECC key via `OpenSSL EVP_DigestSign()`
2. An ML-DSA-87 signature using `pqc_sign()` with domain separation tag `HIGGAION_MIG_V1`

The corresponding `migration_hybrid_verify()` implements *disjunctive* verification: the message is accepted as valid if *either* signature verifies independently. This contrasts with the conjunctive (AND-mode) approach of IETF composite signatures [5].

The rationale is operational: during a gradual migration, some verifiers will support PQC and others will not. Disjunctive verification allows a heterogeneous network to continue operating without coordinated upgrade. As migration progresses and the PQC verification ratio increases, operators can transition to conjunctive or PQC-only policies.

The engine supports three selectable policies:

- **Disjunctive (OR-mode):** Accept if either signature verifies (default during migration)
- **Conjunctive (AND-mode):** Accept only if both verify (maximum security)
- **PQC-preferred:** Accept PQC preferentially; fall back to classical only if PQC signature is absent (not merely invalid)

An extended verification function returns per-component results and maintains verification statistics, including a PQC verification ratio that triggers a downgrade alert when it falls below a configurable threshold.

3.4 Multi-Chain Key Import

Format-specific import adapters handle:

- **Bitcoin:** Base58Check-decoded WIF with checksum verification and secp256k1 public key derivation
- **Ethereum:** Hexadecimal private keys with secp256k1 derivation and Keccak-256 address computation with EIP-55 checksum encoding
- **ECDSA DER:** NIST P-256 and P-384 curves
- **Ed25519:** 32-byte seeds with chain-specific address derivation (Solana base58, Cardano prefixed hex, generic hex)

All imported keys receive a universal identifier: the hexadecimal SHA3-256 hash of the classical public key bytes. This identifier is migration-lifecycle-stable. It does not change as the key transitions through migration states.

3.5 Pre-Migration Self-Test Gating

Before any CLASSICAL → HYBRID transition, the engine performs a cryptographic self-test:

1. Generate ML-DSA-87 signing keypair
2. Generate ML-KEM-1024 key encapsulation keypair
3. Sign a predetermined test message (`PQC_MIGRATION_VERIFY`) with domain separation tag
4. Verify the signature using the generated public key
5. On verification failure: destroy both PQC keypairs and abort the transition

This is an *operation-specific* self-test that validates the PQC implementation for each individual key, distinct from any FIPS 140-3 module-level initialization self-test. It prevents migration to a faulty PQC implementation.

4. Implementation

The engine is implemented as a standalone C library with no dependencies beyond OpenSSL 3.5+. Table 1 summarizes the implementation.

Table 1: Implementation Summary

Component	File	Lines
State machine, WAL, hybrid signing, import adapters	<code>ppc_migration.c</code>	2,459
Test suite (16 lifecycle tests)	<code>test_ppc_migration.c</code>	466
ML-DSA-87 key lifecycle	<code>ppc_crypto.c</code>	244
Keccak-256 (Ethereum addresses)	<code>keccak.c</code>	219
Type definitions and API	<code>ppc_types.h</code> , <code>ppc_migration.h</code> , <code>ppc_crypto.h</code>	958
Total (core, excluding vendored cJSON)		4,346

The library builds with GCC or Clang and links against `libssl` and `libcrypto` from OpenSSL 3.5+, which provides native ML-DSA-87 and ML-KEM-1024 support without requiring third-party PQC providers. Address Sanitizer (ASan) and Undefined Behavior Sanitizer (UBSan) are enabled in the test build.

5. Formal Verification

We employ a quad-tier verification strategy, with each tier targeting different property classes.

5.1 TLA+ Model Checking

A TLA+ specification models the state machine with 3 keys and 8 operations per key. Across 3 TLA+ specifications (state machine, crash recovery, and distributed gossip), the TLC model checker exhaustively checks 16 safety invariants:

Invariant	Property
TypeOK	State values are within the valid enum range
NoSkipToFinal	No direct CLASSICAL $\rightarrow$ FINALIZING transition
RollbackOnlyFromHybridOrFinalizing	Rollback only from HYBRID or FINALIZING
PqcOnlyTerminal	PQC_ONLY is irreversible
ClassicalKeyExists	Classical key exists when state < PQC_ONLY
PqcKeyExists	PQC key exists when state $\geq$ HYBRID

All 16 invariants pass across all reachable states.

5.2 CBMC Bounded Model Checking

CBMC bounded model checking verifies buffer safety, absence of undefined behavior, and state transition correctness with configurable loop unwinding bounds. Properties include array bounds checking, null pointer safety, and enum range validation.

5.3 Frama-C WP Deductive Verification

Frama-C with the WP (Weakest Precondition) plugin verifies 40 ACSL (ANSI/ISO C Specification Language) annotations including function preconditions (**requires**), postconditions (**ensures**), and frame conditions (**assigns**). These contracts prove absence of buffer overflows, correct return value semantics, and memory safety at the function level.

5.4 Coq Mechanized Proofs

A Coq (Rocq 9.1.1) development of 79 theorems (all proved, zero Admitted) mechanically verifies:

- State machine transition correctness and completeness
- WAL ordering guarantees (erasure before irreversible WAL write)
- Disjunctive verification soundness
- Crash recovery safety (no resurrection of erased key material)
- Self-test gating completeness

6. Evaluation

6.1 Test Suite

The test suite comprises 26 tests covering the complete key lifecycle:

Test	Description
<code>test_engine_init_free</code>	Engine initialization and cleanup
<code>test_engine_init_null</code>	Null parameter handling
<code>test_import_bitcoin</code>	Bitcoin WIF key import
<code>test_import_ethereum</code>	Ethereum hex key import with Keccak-256
<code>test_import_ed25519</code>	Ed25519 seed import
<code>test_import_duplicate</code>	Duplicate key rejection
<code>test_import_invalid_key</code>	Invalid key format rejection
<code>test_full_migration_lifecycle</code>	Complete CLASSICAL $\rightarrow$ HYBRID $\rightarrow$ FINALIZING $\rightarrow$ PQC_ONLY
<code>test_migration_rollback</code>	HYBRID $\rightarrow$ CLASSICAL rollback
<code>test_finalize_wrong_state</code>	Finalization from invalid state
<code>test_rollback_from_pqc_only</code>	Rollback rejection from terminal state
<code>test_hybrid_signing</code>	Dual signature generation and verification
<code>test_hybrid_verify_wrong_message</code>	Signature verification failure on tampered message
<code>test_lookup</code>	Key lookup by universal identifier
<code>test_status_report</code>	Migration status reporting
<code>test_migration_counts</code>	Batch migration count tracking

All 26 tests pass with zero ASan/UBSan violations.

6.2 Performance

Table 2 reports execution times on a commodity VPS (single-core, x86-64).

Table 2: Performance (5-run average)

Metric	Time
Full test suite (26 tests)	397 ms
Per-test average	24.8 ms
Minimum run	373 ms
Maximum run	446 ms

The per-test average of 24.8 ms includes ML-DSA-87 key generation, signing, and verification. This is dominated by the PQC cryptographic operations, which are performed by OpenSSL 3.5’s native ML-DSA implementation. The migration state machine logic itself (state transitions, WAL I/O, key import parsing) adds negligible overhead.

7. Limitations and Honest Assessment

We believe transparency about the limitations of this work is essential for its responsible evaluation.

AI-Assisted Development. The implementation was developed with AI coding assistants under the direction of the author. While all code was reviewed, tested, and formally verified, readers should be aware that the development process was AI-assisted rather than purely manual. The formal verification artifacts were developed alongside the implementation by the same author, not by an independent verification team.

No Independent Security Audit. The system has not undergone independent third-party security audit by a professional security firm. The formal verification provides strong guarantees about specific property classes (state machine correctness, memory safety, crash recovery ordering), but it does not substitute for a comprehensive adversarial security review.

Scope Limitation. The standalone library has been verified in isolation. Integration into a production distributed ledger system, with network adversaries, Byzantine fault tolerance requirements, and real-world operational pressures, has not been validated. The disjunctive verification policy, while operationally convenient during migration, relaxes security guarantees relative to conjunctive verification and requires careful policy transition planning.

Solo Practitioner. This work was conducted by a single independent researcher without institutional affiliation. While this demonstrates that PQC migration tooling is accessible to individual practitioners, it also means the work has not benefited from the multi-person code review and adversarial testing typical of institutional security projects.

Performance Characterization. The benchmarks reported were collected on a single commodity VPS and should not be considered comprehensive performance characterization. Production deployments would require profiling under realistic load conditions with concurrent key operations.

8. Conclusion and Future Work

We have presented a PQC Wallet Migration Engine that addresses five specific gaps in the current PQC migration landscape: crash-recoverable state transitions, disjunctive hybrid verification, multi-chain key import, pre-migration self-test gating, and quad-tier formal verification. The system is implemented in 2,922 lines of C (core logic, excluding vendored dependencies and tests) and builds against OpenSSL 3.5+ with native ML-DSA-87 and ML-KEM-1024 support.

Future work includes: (1) independent third-party security audit by a qualified firm; (2) integration with hardware security modules for PQC key storage; (3) production deployment and performance characterization under realistic load; (4) extension of formal verification to cover concurrent multi-thread access; and (5) investigation of constrained-device adaptations for IoT environments.

The source code, formal verification artifacts, and test suite are available for review from the author.

References

- [1] P. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th IEEE FOCS*, 1994, pp. 124-134.
- [2] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, August 2024.
- [3] NIST, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, August 2024.

- [4] Office of Management and Budget, “Memorandum M-23-02: Migrating to Post-Quantum Cryptography,” 2022.
- [5] M. Ounsworth et al., “Composite ML-DSA for Use in Internet PKI,” Internet-Draft draft-ietf-lamps-pq-composite-sigs, IETF, 2024.
- [6] D. Stebila et al., “Hybrid Signature Spectrums,” Internet-Draft draft-ietf-pquip-hybrid-signature-spectrums, IETF, 2024.
- [7] Y. Sheffer et al., “Post-Quantum Traditional (PQ/T) Hybrid Dual Certificates,” Internet-Draft draft-yusef-tls-pqt-dual-certs, IETF, 2024.
- [8] IBM, “IBM Quantum Safe Explorer, Remediator, and Migration Orchestrator,” 2024. [Online]. Available: <https://www.ibm.com/quantum/quantum-safe>
- [9] Thales, “Luna HSM Key Rotation and Domain Validation,” 2024.
- [10] The QRL Foundation, “Quantum Resistant Ledger,” 2024. [Online]. Available: <https://www.theqrl.org>
- [11] IOTA Foundation, “IOTA Identity 1.7 Beta: ML-DSA Hybrid Signatures,” 2025.

Provisional Patent Application filed March 9, 2026, USPTO.

Development methodology: AI-assisted implementation under author direction. See Section 7 for complete disclosure.